

1. Make Github Repo
2. Make virtual environment and activate python environment

`python -m venv .venv` --- Creating a virtual environment

`.venv\Scripts\activate` --- Activate the environment

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Valin\Downloads\cloud-native-devops-platform> python --version
Python 3.13.7
PS C:\Users\Valin\Downloads\cloud-native-devops-platform> pip --version
pip 25.3 from C:\Users\Valin\AppData\Roaming\Python\Python313\site-packages\pip (python 3.13)
PS C:\Users\Valin\Downloads\cloud-native-devops-platform> python -m venv .venv
PS C:\Users\Valin\Downloads\cloud-native-devops-platform> .venv\Scripts\activate
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform>
```

3. Install FastAPI

`pip install fastapi uvicorn` --- Install dependencies into that environment.

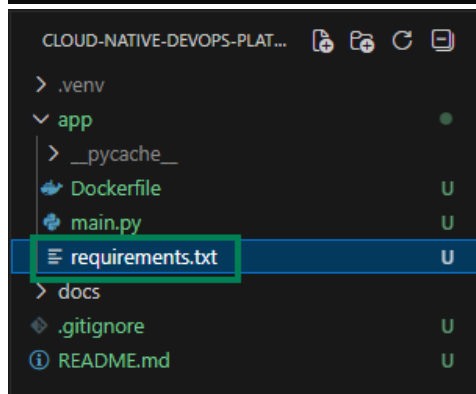
FastAPI = API framework/application

Uvicorn = server that runs the FastAPI application

`pip freeze > requirements.txt` -- Record all Python packages currently installed in this environment in a file called requirements.txt.

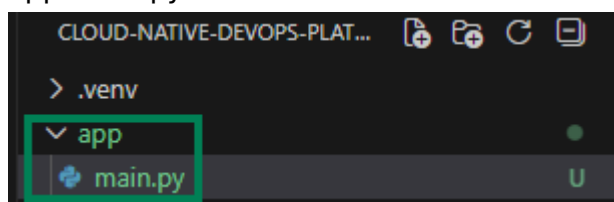
```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> pip install fastapi uvicorn
Collecting fastapi
  Downloading fastapi-0.141.1-py3-none-any.whl.metadata (27 kB)
Collecting uvicorn
  Downloading uvicorn-0.53.0-py3-none-any.whl.metadata (6.6 kB)
```

```
PS C:\Users\Valin\Downloads\cloud-native-devops-platform\app> pip freeze > requirements.txt
PS C:\Users\Valin\Downloads\cloud-native-devops-platform\app>
```



4. Create the first application

app/main.py



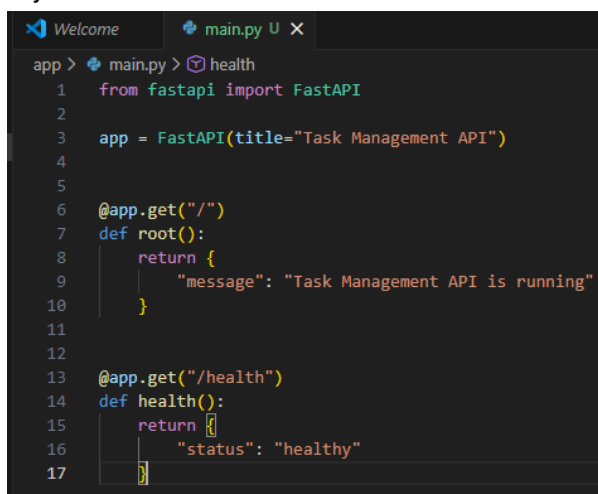
Content:

```
from fastapi import FastAPI
```

```
app = FastAPI(title="Task Management API")
```

```
@app.get("/")
def root():
    return {
        "message": "Task Management API is running"
    }
```

```
@app.get("/health")
def health():
    return {
        "status": "healthy"
    }
```

A screenshot of a code editor window with a dark theme. The window has two tabs: 'Welcome' and 'main.py'. The 'main.py' tab is active, showing the Python code for a FastAPI application. The code is identical to the blocks above, including the imports, the FastAPI app creation, and the two endpoint functions. Line numbers 1 through 17 are visible on the left side of the editor.

uvicorn main:app --reload (first, move to folder app) --- Running FastAPI with Uvicorn

Notes: main:app means a FastAPI object named `app`

--reload : Automatically reloads when the code changes.

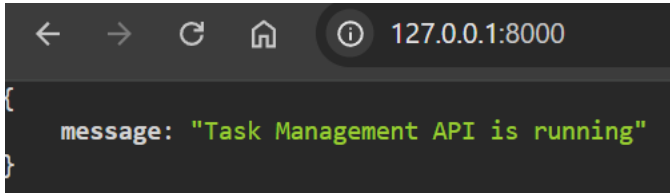
In conclusion, it means : 'Run the FastAPI application from `main.py` using the Uvicorn server, with automatic restarts whenever the code changes.'

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> cd
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> dir

Directory: C:\Users\Valin\Downloads\cloud-native-devops-platform

Mode                LastWriteTime         Length Name
----                -
d-----          9/14/2026   4:55 PM             .venv
d-----          9/14/2026   5:11 PM             app
d-----          9/14/2026   4:41 PM             docs
-a-----          9/14/2026   4:44 PM              0 .gitignore
-a-----          9/14/2026   4:43 PM              0 README.md
-a-----          9/14/2026   5:05 PM          496 requirements.txt

(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> cd app
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform\app> uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\Users\Valin\Downloads\cloud-native-devops-platform\app']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [34484] using StatReload
INFO: Started server process [39276]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:64570 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:64570 - "GET /favicon.ico HTTP/1.1" 404 Not Found
```



Notes: Add `.venv/` to `.gitignore`. The next contributor can clone the project from GitHub, create their own virtual environment, and install the required dependencies using `requirements.txt`.

git clone ...

cd cloud-native-devops-platform

python -m venv .venv

.\venv\Scripts\Activate.ps1

pip install -r requirements.txt

python -m uvicorn main:app --reload

5. Try /health

Open <http://127.0.0.1:8000/health>

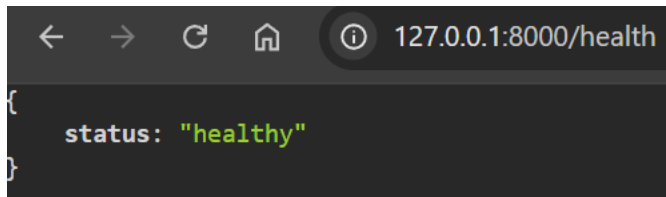
This is actually simple, but this endpoint will be very useful for Kubernetes later.

Kubernetes can then ask: "Is your application still alive?"

Application:

/status → healthy

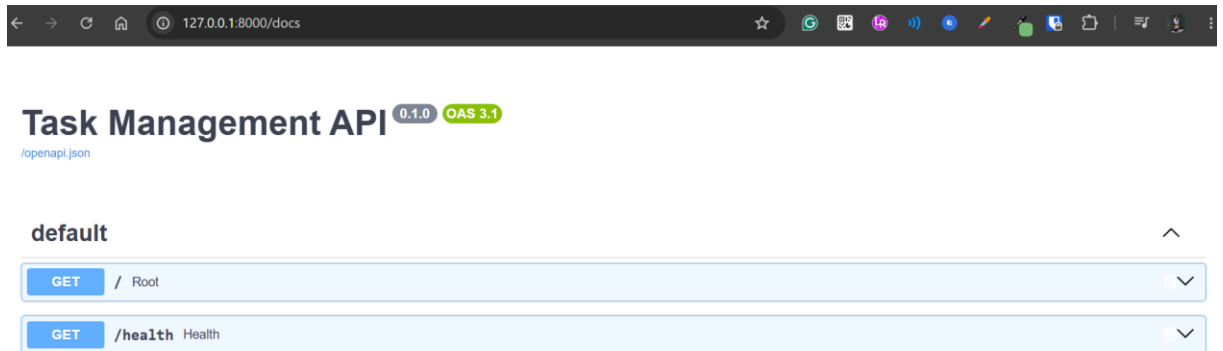
If it's not healthy, Kubernetes can take action.



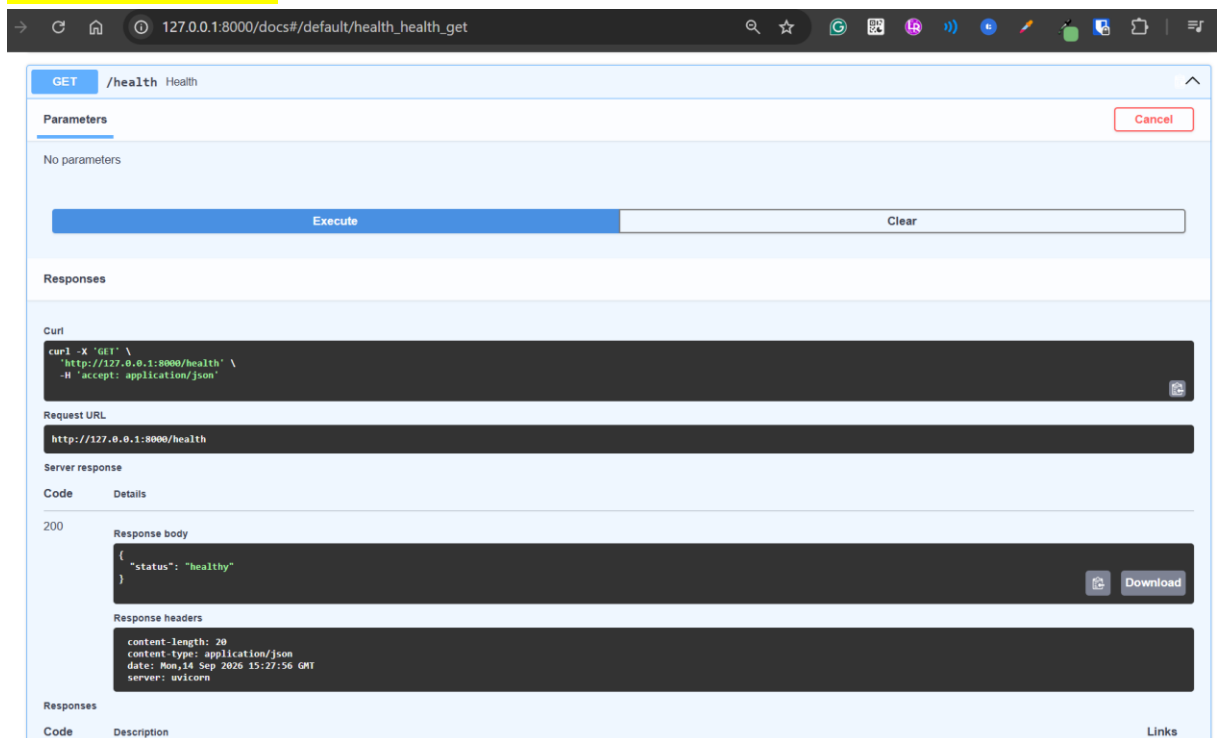
6. Look at FastAPI Swagger

Open <http://127.0.0.1:8000/docs>

We will see interactive API documentation



Try it out → Execute



7. Create the Task API.

Now we start building the actual application.

Change the main.py content become:

from fastapi import FastAPI

from pydantic import BaseModel

```
app = FastAPI(title="Task Management API")
```

```
class Task(BaseModel):  
    title: str  
    completed: bool = False
```

```
tasks = []
```

```
@app.get("/")  
def root():  
    return {  
        "message": "Task Management API is running"  
    }
```

```
@app.get("/health")  
def health():  
    return {  
        "status": "healthy"  
    }
```

```
@app.get("/tasks")  
def get_tasks():  
    return tasks
```

```
@app.post("/tasks")  
def create_task(task: Task):  
    new_task = {  
        "id": len(tasks) + 1,  
        "title": task.title,  
        "completed": task.completed  
    }
```

```
tasks.append(new_task)
```

```
return new_task
```

```
Welcome | main.py U X
app > main.py > health
5
6
7 class Task(BaseModel):
8     title: str
9     completed: bool = False
10
11
12 tasks = []
13
14
15 @app.get("/")
16 def root():
17     return {
18         "message": "Task Management API is running"
19     }
20
21
22 @app.get("/health")
23 def health():
24     return {
25         "status": "healthy"
26     }
27
28
29 @app.get("/tasks")
30 def get_tasks():
31     return tasks
32
33
34 @app.post("/tasks")
35 def create_task(task: Task):
36     new_task = {
37         "id": len(tasks) + 1,
38         "title": task.title,
39         "completed": task.completed
40     }
41
42     tasks.append(new_task)
43
44     return new_task
```

Restart the server if needed

Open <http://127.0.0.1:8000/docs>

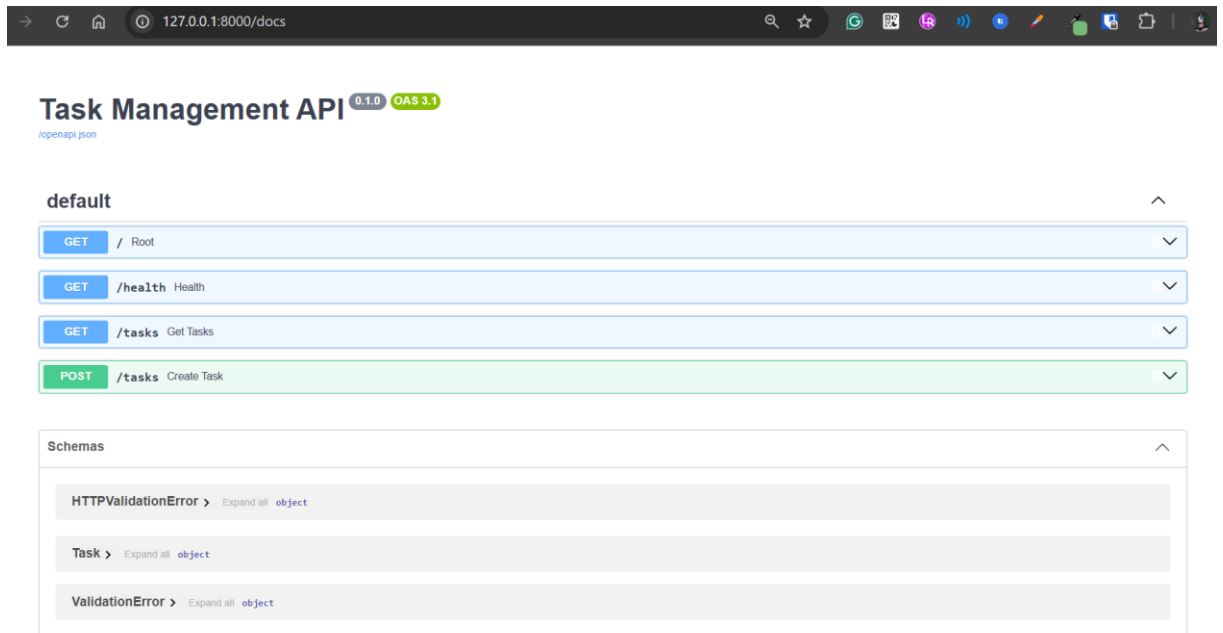
Now you will see:

GET /

GET /health

GET /tasks

POST /tasks

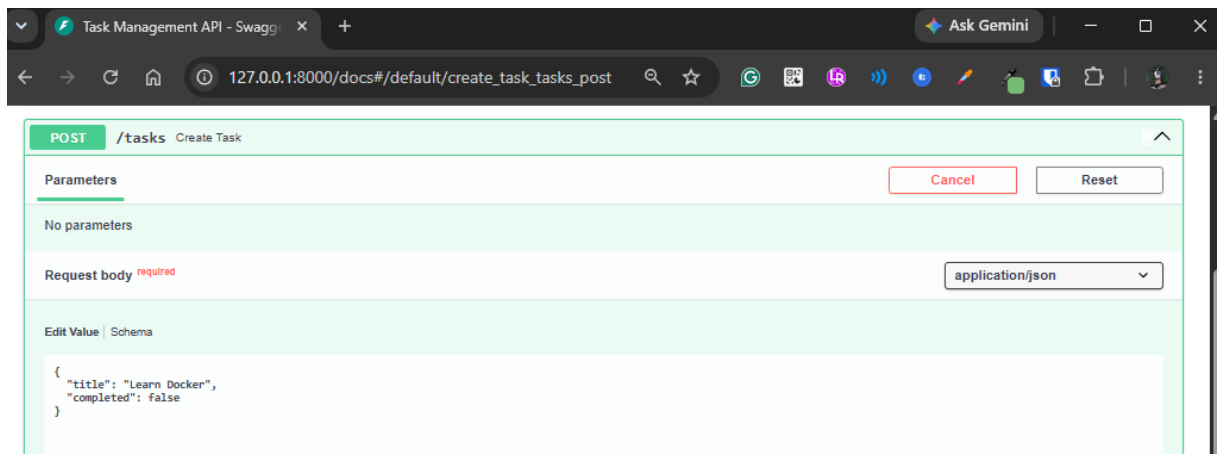


Now, let's play on the Swagger:

POST /tasks

Click **Try it out** and enter:

```
{  
  "title": "Learn Docker",  
  "completed": false  
}
```



Click **Execute**.

Response:

Task Management API - Swagger x + Ask Gemini

127.0.0.1:8000/docs#/default/create_task_tasks_post

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/tasks' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "title": "Learn Docker",
    "completed": false
  }'
```

Request URL

http://127.0.0.1:8000/tasks

Server response

Code Details

200

Response body

```
{
  "id": 1,
  "title": "Learn Docker",
  "completed": false
}
```

Response headers

```
content-length: 49
content-type: application/json
date: Mon, 14 Sep 2026 15:41:37 GMT
server: uvicorn
```

And then.. try **GET /tasks**

Task Management API - Swagger x + Ask Gemini

127.0.0.1:8000/docs#/default/get_tasks_tasks_get

GET /tasks Get Tasks

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/tasks' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/tasks

Server response

Code Details

200

Response body

```
[
  {
    "id": 1,
    "title": "Learn Docker",
    "completed": false
  }
]
```

Response headers

```
content-length: 51
content-type: application/json
date: Mon, 14 Sep 2026 15:46:09 GMT
server: uvicorn
```

Now... we already have REST API!!

REST = Representational State Transfer.

REST is neither a library nor a programming language. REST is a **design style** or set of rules for creating HTTP-based APIs.

An **API** is an interface or **intermediary** that enables software to communicate with other software.

Browser / Frontend

↓
GET /users
▼

REST API

↓
▼

FastAPI

↓
▼

Database

8. Dockerize

Now, we move on to DevOps.

Our application that we made now:

Python

↓

FastAPI

↓

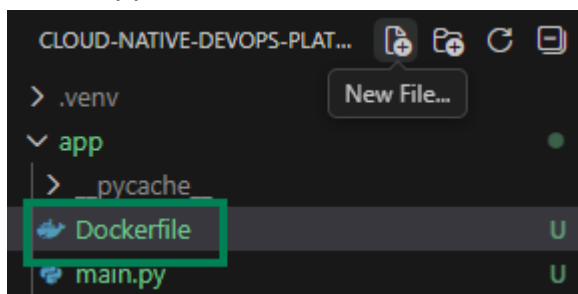
Works on my laptop

The problem: If it is moved to the server, will the environment be the same?

Docker helps to solve that problem.

Now, make Dockerfile. A **Dockerfile** is a set of instructions or a recipe for Docker on how to build an image containing everything our application needs to run.

Inside app/Dockerfile



Write:

FROM python:3.12-slim

WORKDIR /app

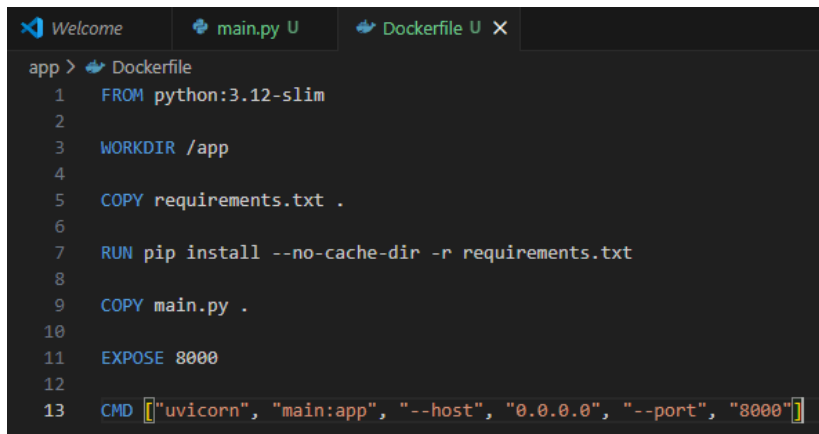
COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY main.py .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]



```
app > Dockerfile
1 FROM python:3.12-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6
7 RUN pip install --no-cache-dir -r requirements.txt
8
9 COPY main.py .
10
11 EXPOSE 8000
12
13 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

uvicorn main:app --host 0.0.0.0 --port 8000

This means:

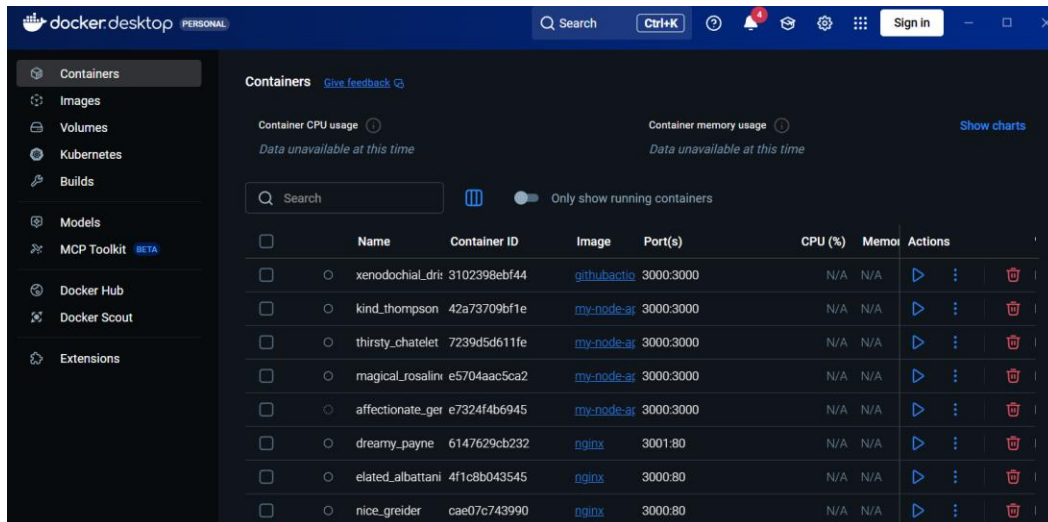
"Uvicorn, listen for requests on all network interfaces available to this environment, on port 8000."

So, 0.0.0.0 is more like: "Listen at every door."

Not: "My website address is 0.0.0.0."

9. Build Docker Image

Make sure Docker Desktop already running!



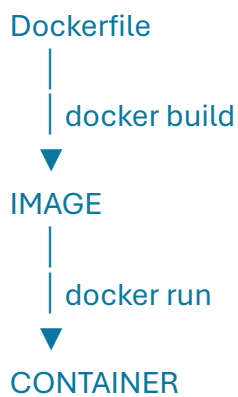
From **app** folder:

1. `docker build -t task-management-api .`
2. `docker images`

and then we should see **task-management-api**

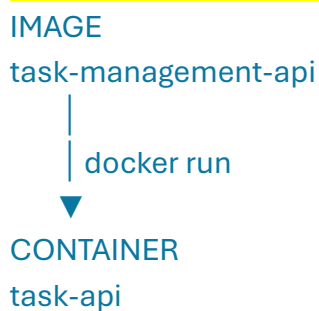
```
PS C:\Users\Valin\Downloads\cloud-native-devops-platform\app> docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
task-management-api  latest     77ce88add46e  About a minute ago  238MB
gitlab-cicd         latest     d5fc6bfabb4f  7 months ago   192MB
hello-node          latest     e4d6439eb15e  7 months ago   192MB
```

Important concept:



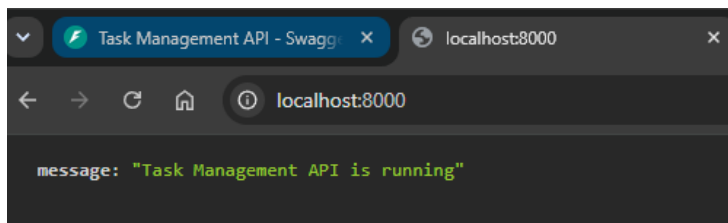
10. Run the container

`docker run -d -p 8000:8000 --name task-api task-management-api`



and now open <http://localhost:8000>

```
MINGW64:/c/Users/Valin
Valin@DESKTOP-VMJSNLD MINGW64 ~
$ docker run -d -p 8000:8000 --name task-api task-management-api
65d1b9aa870bd700906992f09c2533aa657695b472fdf5f22a26f587d0ed6a3e
```

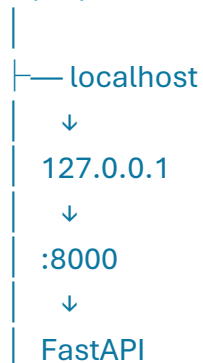


So, we just running the application in Docker

Visualization:

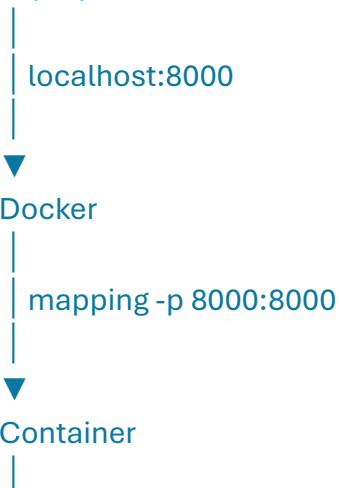
a. Without Docker:

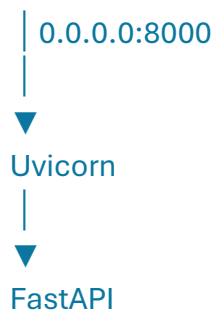
Laptop



With Docker:

Laptop





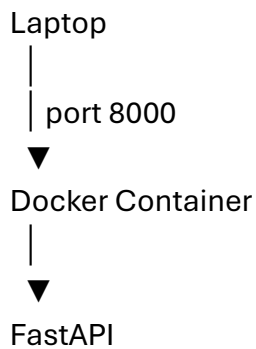
11. Test container

docker ps

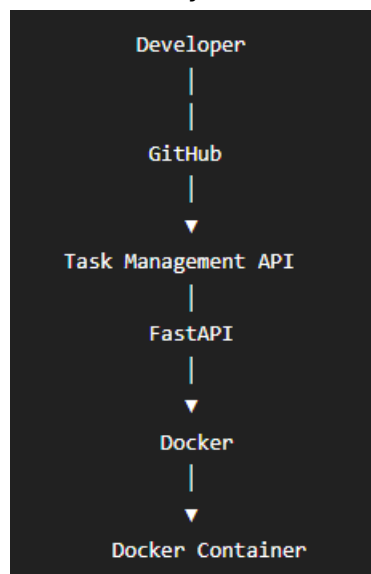
```
va1in@DESKTOP-VMJ5NLD MINGW64 ~  
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
65d1b9aa870b	task-management-api	"uvicorn main:app --..."	2 minutes ago	Up 2 minutes	0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp	task-api
8946a792e8d4	mariadb:10s	"docker-entrypoint.s..."	9 months ago	Up 24 minutes (healthy)	0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp	mariadb

It means,



This is the day 1. So far we made:



Push to Github

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        README.md
        app/
        docs/

nothing add to track; use "git add" to track
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> git add .
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> git commit -m "feat: create FastAPI task management API with Docker"
[main (root-commit) 6e1efe8] feat: create FastAPI task management API with Docker
6 files changed, 147 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 README.md
 create mode 100644 app/Dockerfile
 create mode 100644 app/main.py
 create mode 100644 app/requirements.txt
 create mode 100644 docs/deployment.md
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> git remote add origin https://github.com/valinfernanda/cloud-native-devops-platform.git
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> git push -u origin main
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 12 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (10/10), 2.16 KiB | 441.00 KiB/s, done.
Total 10 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/valinfernanda/cloud-native-devops-platform.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

valinfernanda / cloud-native-devops-platform

Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

cloud-native-devops-platform Public

main 1 Branch 0 Tags

Go to file Add file Code

valinfernanda feat: create FastAPI task management API with Docker 6e1efe8 · 2 minutes ago 1 Commit

app	feat: create FastAPI task management API with Docker	2 minutes ago
docs	feat: create FastAPI task management API with Docker	2 minutes ago
.gitignore	feat: create FastAPI task management API with Docker	2 minutes ago
README.md	feat: create FastAPI task management API with Docker	2 minutes ago

Self reflection....

1. What is an API?

An API is an interface or set of rules that allows one piece of software to communicate with another.

2. What is FastAPI?

A Python web framework for building APIs and web applications.

3. What is Docker?

Docker is a tool for creating images and running those images as containers.

Dockerfile

↓

docker build

↓

Image

↓

docker run

↓

Container

↓

Application running

4. What is the difference between a Docker image and a Docker container?

A Docker image is a template or package containing an application along with the environment and dependencies required to run it. **A container** is a running instance of an image.

5. Why do our applications need to be containerized?

To make it portable and so that it can be run in many environments

6. Why does our DevOps project require an application?

Well, if there's no application, what exactly is there to run and monitor, right?

At the end, all those tools are used to manage the application lifecycle.